# Deep Learning with PyTorch

*Image Classification with a CNN and Sentiment Analysis with an RNN*

**Vishal Kumar** | MCA Student, Jaipur National University
GitHub: https://github.com/vishu56096-ctrl | LinkedIn: https://www.linkedin.com/in/vishal-kumar-7a4003393/

**Abstract.** This report documents a repository of two PyTorch deep-learning projects: a convolutional neural network (CNN) that classifies CIFAR-10 images, and a recurrent-network model that classifies the sentiment of 50,000 IMDB movie reviews. It explains every pipeline stage (data preparation, model design, training and evaluation), reports the real results (74.75% and 85.39% test accuracy), and analyses what the numbers mean, where each notebook can be improved, and which skills the work demonstrates.

### Contents

1. Repository Overview . . . . . 1

2. Project 1: CNN for CIFAR-10 Image Classification . . . . . 1

3. Project 2: RNN for IMDB Sentiment Analysis . . . . . 3

4. Comparison, Skills and Conclusion . . . . . 4

## 1. Repository Overview

The repository is a progressive learning path in deep learning. Notebook 01 teaches how a network learns from **images** through convolution; notebook 02 teaches how text is cleaned, converted to numbers and classified by a **recurrent** layer. Both follow the same workflow: load data, prepare it, define a *nn.Module*, train with a loss and an optimiser, then evaluate on unseen data. Python 3.13.9, PyTorch, torchvision, pandas, NLTK and scikit-learn are used.

*Listing 1: Repository layout implied by the notebook paths.*

```
<repo>/
|-- <notebook folder>/        (notebooks read their data from ../data/...)
|   |-- 01_cnn_cifar10_image_classification.ipynb
|   `-- 02_rnn_imdb_sentiment_analysis.ipynb
`-- data/
    |-- cnn/    CIFAR-10, downloaded automatically by torchvision
    `-- rnn/    IMDB Dataset.csv
```

*Table 1: The two projects side by side.*

| | 01 CNN: CIFAR-10 | 02 RNN: IMDB sentiment |
|---|---|---|
| **Task** | 10-class image classification | Binary text classification (positive / negative) |
| **Data** | 60,000 colour images, 32x32 (50,000 train, 10,000 test) | 50,000 reviews; 49,582 after removing duplicates |
| **Input to model** | Normalised 3x32x32 tensor | 6,000-value TF-IDF vector per review |
| **Model** | 3 conv blocks + 2 linear layers, 620,362 parameters | nn.RNN (hidden 64) + linear, 388,289 parameters |
| **Loss / optimiser** | CrossEntropyLoss / Adam | BCELoss / Adam (lr 0.001) |
| **Batch size / epochs** | 64 / 10 | 64 / 10 |
| **Test accuracy** | **74.75%** (chance: 10%) | **85.39%** (chance: 50%) |

## 2. Project 1: CNN for CIFAR-10 Image Classification

### 2.1 Problem and data

CIFAR-10 contains 60,000 small colour images (32x32 pixels) in 10 classes such as airplane, cat, ship and truck: 50,000 for training and 10,000 for testing. Each image becomes a tensor of shape 3x32x32 (channels x height x width). The task is hard because the images are tiny and objects vary in position, colour and background.

## 2.2 Data pipeline

*ToTensor* converts each image to floats in [0, 1] and puts channels first. *Normalize((0.5,0.5,0.5), (0.5,0.5,0.5))* then computes (x - 0.5) / 0.5, moving values to [-1, 1] so inputs are centred on zero, which keeps gradients well behaved. Data is served by *DataLoader* in batches of 64 (782 training batches and 157 test batches per pass); only the training loader is shuffled.

## 2.3 Architecture

*Table 2: Layer-by-layer shapes and parameter counts (computed from the code).*

| Layer | Output shape | Parameters |
|---|---|---|
| Input image | 3 x 32 x 32 | 0 |
| Conv2d(3 to 32, 3x3, padding 1) + ReLU | 32 x 32 x 32 | 896 |
| MaxPool2d(2) | 32 x 16 x 16 | 0 |
| Conv2d(32 to 64, 3x3, padding 1) + ReLU | 64 x 16 x 16 | 18,496 |
| MaxPool2d(2) | 64 x 8 x 8 | 0 |
| Conv2d(64 to 128, 3x3, padding 1) + ReLU | 128 x 8 x 8 | 73,856 |
| MaxPool2d(2) | 128 x 4 x 4 | 0 |
| Flatten | 2,048 | 0 |
| Linear(2048 to 256) + ReLU | 256 | 524,544 |
| Linear(256 to 10) (class scores) | 10 | 2,570 |
| **Total** | | **620,362** |

A 3x3 convolution with padding 1 keeps the width and height, and each max-pool halves them (32, 16, 8, 4). The channel count doubles (32, 64, 128) as the image shrinks: early filters respond to edges and colours, deeper filters combine them into textures and object parts. The feature map is flattened to 2,048 numbers and passed through two linear layers. About 85% of all parameters sit in the first linear layer. The final layer outputs 10 raw scores (logits); no softmax is added because *CrossEntropyLoss* applies it internally.

*Listing 2: The CNN (condensed from the notebook).*

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2))
        self.fc_layers = nn.Sequential(
            nn.Linear(4 * 4 * 128, 256), nn.ReLU(), nn.Linear(256, 10))
    def forward(self, x):
        x = self.conv_layers(x)
        return self.fc_layers(x.view(x.size(0), -1))
```

## 2.4 Training and evaluation

The notebook trains for 10 epochs with *CrossEntropyLoss* and *Adam* (default learning rate 0.001). Each batch runs the standard five steps: *zero_grad* clears old gradients, the forward pass produces scores, the loss measures the error, *backward* computes gradients, and *step* updates the weights. The reported epoch loss is the average over batches. For evaluation the model is switched to *eval()*, gradients are disabled with *no_grad()*, and the predicted class is the index of the highest score (*torch.max*).

**Result:** the model classifies 74.75% of the 10,000 test images correctly (7,475 images), far above the 10% chance level.

## 2.5 Analysis and observations

Three points stand out from the code and the logged numbers:

- **Overfitting is likely.** Training loss falls from 0.136 to 0.064, a loss at which the network usually classifies most training images correctly, while test accuracy is only 74.75%. Training accuracy was not measured, so the size of the gap is an estimate. The network has no augmentation, dropout, batch normalisation or weight decay, and it has a large 524,544-parameter linear layer that can memorise images.

- **The first logged loss looks too low.** A freshly initialised 10-class network starts near 2.3 (the natural log of 10), yet epoch 1 shows 0.136. This suggests the training cell was run more than once, so the log shows a later run. For a clean record, re-create the model and train once.
- **Only overall accuracy is reported**, with no validation split and no per-class results.

*Table 3: Planned improvements for the CNN notebook.*

| Improvement | Why it helps |
|---|---|
| Random crop and horizontal flip (augmentation) | Shows the network more variety and reduces memorising |
| BatchNorm and Dropout | Stabilise training and regularise the large linear layer |
| Per-channel mean/std, weight decay, learning-rate schedule | Better conditioned optimisation and generalisation |
| Validation split with early stopping; confusion matrix | Choose the stopping point honestly and see which classes confuse (cat vs dog) |
| Train on GPU; call model(x) instead of model.forward(x) | Faster experiments and the standard PyTorch style (hooks are applied) |

# 3. Project 2: RNN for IMDB Sentiment Analysis

## 3.1 Problem and data

The goal is to decide whether a movie review is **positive** or **negative**. The CSV has two columns (*review*, *sentiment*) and 50,000 rows with no missing values. Dropping 418 exact duplicates leaves **49,582** reviews. The original dataset is balanced, so random guessing scores about 50%.

## 3.2 Text pre-processing pipeline

*Table 4: Pre-processing steps and what they actually do.*

| Step | What the code does | Observed effect |
|---|---|---|
| **1 Lowercase** | str.lower() | 'Great' and 'great' become one word |
| **2 Remove URLs** | regex http\S+ | Links removed |
| **3 Remove punctuation** | regex keeps only A-Z, a-z, 0-9, spaces | Also deletes the angle brackets of HTML tags |
| **4 Remove HTML tags** | regex <.*?> | Does nothing here: step 3 already removed the brackets, so 'br br' stays in the text |
| **5 Remove stopwords** | tokenise; for each stopword, text.replace(word, '') | Deletes stopword letters inside other words, e.g. 'wonderful' becomes 'wderful' |
| **6 Stemming** | PorterStemmer on each token | Merges forms such as running and run |
| **7 Label encoding** | LabelEncoder | negative = 0, positive = 1 |
| **8 TF-IDF** | TfidfVectorizer(max_features=6000) | Sparse matrix 49,582 x 6,000, 4,160,624 non-zero values (1.4% filled) |

**TF-IDF** turns each review into a vector of 6,000 numbers. A word scores high when it is frequent in that review but rare across all reviews, so distinctive words such as 'awful' or 'brilliant' carry more weight than common ones. The labels are split 80/20 with *random_state=42* into 39,665 training and 9,917 test reviews, converted to dense float tensors (about 1.9 GB for the training matrix as float64) and served in batches of 64 (620 training batches per epoch).

## 3.3 Model

The network is one *nn.RNN* layer (6,000 inputs, 64 hidden units) followed by *Linear(64 to 1)*. A recurrent layer updates a hidden state at every time step: $h_t = \tanh(W_{ih} x_t + b_{ih} + W_{hh} h_{t-1} + b_{hh})$. The output of the last step goes to the linear layer, a sigmoid converts the score to a probability, and *BCELoss* compares it with the 0/1 label. Parameters: 6,000 x 64 + 64 x 64 + 64 + 64 for the RNN, plus 65 for the output layer, total **388,289**. The notebook defines the class twice; the first version failed because *nn.RNN* returns a tuple (output, hidden), and the second version unpacks it correctly.

*Listing 3: The final RNN class (condensed from the notebook).*

```
class RNN(nn.Module):
    def __init__(self, input_size, hidden_size=64, num_layers=1):
        super().__init__()
        self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, 1)
    def forward(self, x):                  # x: (batch, 1, 6000)
        out, hidden = self.rnn(x)
        return self.fc(out[:, -1, :])      # last time step -> one score
```

### 3.4 Training and results

Training uses *Adam* (learning rate 0.001) for 10 epochs. The loss drops fast in epoch 1 (0.4046 to 0.2577), then improves slowly, ending at 0.2054; by the last epochs each epoch gains less than 0.002, so more epochs would add little. On the held-out test set the model classifies **85.39%** of reviews correctly (8,468 of 9,917).

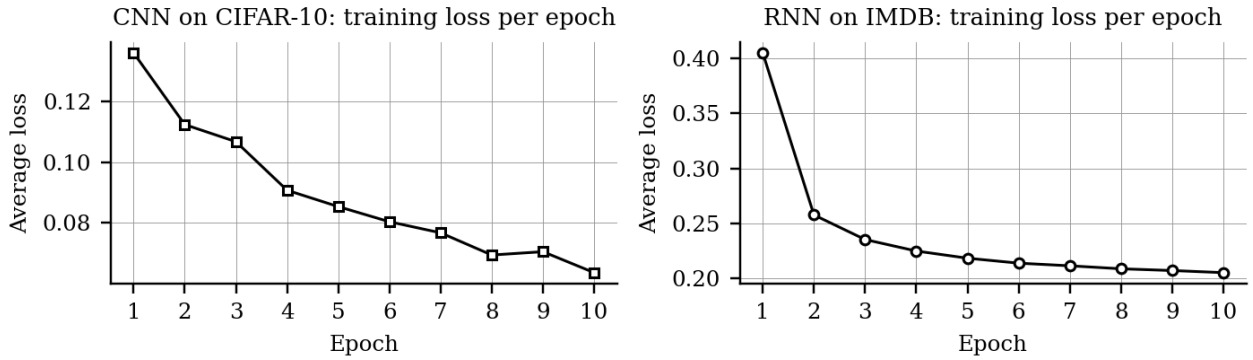



*Figure 1: Average training loss per epoch, from the notebook output.*

### 3.5 Findings and fixes

*Table 5: Code review of the RNN notebook.*

| Finding | Impact | Fix |
|---|---|---|
| The RNN sees sequences of length 1: unsqueeze(1) feeds each whole review as a single step | No recurrence over words happens, so the layer behaves like a dense layer with tanh | Tokenise to word-id sequences, add an Embedding layer, use LSTM or GRU |
| Stopword removal uses text.replace on raw text | Words are corrupted, though consistently, so most signal survives (85% accuracy) | Keep tokens not in a stopword set: ' '.join(w for w in tokens if w not in stop) |
| HTML removal runs after punctuation removal | 'br' tokens remain in reviews | Remove HTML tags first, then punctuation |
| TF-IDF is fitted on all reviews before the split | Small information leak from test data | Fit on training data only, then transform the test data |
| Sigmoid applied outside the model with BCELoss | Works, but less numerically stable | Use BCEWithLogitsLoss on raw scores |
| Only accuracy is reported | Hides error types | Add precision, recall, F1 and a confusion matrix |

A useful reference point is a logistic-regression model on the same TF-IDF features, which is commonly reported to reach close to 90% on this dataset. An LSTM or GRU on real word sequences, or a fine-tuned transformer, is the natural next step.

## 4. Comparison, Skills and Conclusion

The two notebooks show two ways of turning raw data into numbers a network can learn from: pixels need only scaling, while text needs a multi-step cleaning and vectorising pipeline, and that pipeline decides the quality of the result as much as the model does. The CNN reaches 74.75% on a 10-way task and probably overfits; the RNN reaches 85.39% on a binary task, but its recurrent layer is not yet used as intended.

**Skills demonstrated:** building *nn.Module* models; computing layer shapes and parameter counts; writing training and evaluation loops; using DataLoaders and TensorDatasets; text cleaning with regex and NLTK; stemming and TF-IDF with scikit-learn; reading loss curves; diagnosing overfitting and data leakage; and critically reviewing one's own code. The next steps are the fixes in Tables 3 and 5, then re-running both notebooks and updating this report with the new

results.